CSA06 – DESIGN AND ANALYSIS OF ALGORITHMS

CO2 – Debugging and Optimisation Task

Student Name	Mohammed Affan M
Register Number	192572169
Scenario	SentinelOps SOC Alert-Triage Module
Course Outcome	Brute-force sorting, searching, string matching and closest-pair problems

Note: The explanations and corrected code below are based on the supplied assessment sheet. Timing figures are not fabricated: the operation-count table gives theoretical counts/formulas, and the included test harness can be run to collect machine-specific measurements.

1. Aim

To identify and fix the nine defects in the supplied Python module, explain each root cause, test boundary cases, and provide a more efficient version suitable for alert triage.

2. Algorithms

A. Stable sorting alerts by severity (highest first)

Algorithm: Use a stable sort with the severity as the descending key. Stability keeps alerts with equal severity in their original arrival order.

• Start.

• Sort alerts by severity in descending order using a stable sorting method.

• Return the sorted list.

B. Sequential blocklist search

• Check each IP address in the blocklist.

• If an address matches the requested IP, return True immediately.

• If the loop ends without a match, return False.

C. Find all occurrences of a signature

• Let n be the log length and m be the pattern length.

• Check every valid starting position from 0 through n – m, inclusive.

• Compare characters until a mismatch or a complete match.

• Record the start position of each complete match; advance by one to allow overlaps.

D. Closest pair of login locations

• If fewer than two events exist, return (None, infinity).

• Use the divide-and-conquer closest-pair algorithm: sort points by x-coordinate, split into halves, and solve each half.

• Let delta be the smaller half-distance; inspect points in the vertical strip within delta of the split.

• Compare only nearby strip points in y-order and update the best squared distance.

• Return the pair of original event indices and the square root of the best squared distance.

3. Defects found (nine)

No.	Function / line	Defect and correction
1	sort_alerts, line 39	j > 0 skips index 0; use j >= 0 in insertion sort, or use Python's stable sorted().
2	sort_alerts, line 42	Writing key to alerts[j] can overwrite an item after shifting. In insertion sort, place it at alerts[j+1].
3	is_blocked, line 48	A matching blocked IP returns False. It must return True when a match is found.

No.	Function / line	Defect and correction
4	is_blocked, line 49	The else returns False after the first non-match, so later addresses are never checked. Return False only after the loo
5	find_signature, line 55	range(n-m) excludes the final valid start and yields no checks when n == m. Use range(n-m+1), after handling m > n.
6	find_signature, line 59	After break, j can equal m-1 even though the last character mismatched, causing a false match. Use an explicit full-m
7	closest_logins, line 64	min_dist = 0 prevents any nonnegative distance from being smaller. Initialise best squared distance to infinity.
8	closest_logins, line 67	Comparing every j includes the same event and repeats pairs. Compare only distinct pairs (j > i), or use divide and co
9	closest_logins, line 70	The code selects the largest distance using d > min_dist. The closest pair requires updating when d < best distance.

The faulty code may also fail on an empty pattern because j is never assigned inside the inner loop. The corrected implementation explicitly handles an empty pattern.

4. Root causes and verification

A. Sorting trace

Input: A1(3), A2(9), A3(5), A4(9), A5(1). The corrected descending stable order is A2(9), A4(9), A3(5), A1(3), A5(1). A2 remains before A4 because both have severity 9 and A2 arrived first.

Pass	Key	Result after insertion
1	A2(9)	A2(9), A1(3), A3(5), A4(9), A5(1)
2	A3(5)	A2(9), A3(5), A1(3), A4(9), A5(1)
3	A4(9)	A2(9), A4(9), A3(5), A1(3), A5(1)
4	A5(1)	A2(9), A4(9), A3(5), A1(3), A5(1)

B. Blocklist checks

For blocklist [10.0.0.5, 10.0.0.9, 172.16.4.2], the correct outputs for queries [10.0.0.9, 10.0.0.5, 8.8.8.8] are [True, True, False]. An empty blocklist must return False.

C. Signature matching trace

Log = FAILFAIXFAIL; pattern = FAIL. Valid starting positions are 0 through 8. Matches start at indices 0 and 8 (zero-based). The final match is found only when the last valid start is included. A pattern longer than the log returns an empty list. An empty pattern is defined here to match at every boundary from 0 through n.

D. Closest login locations

The sample events are in kilometres. The closest pair is event indices (0, 5): (2,3) and (3,4), with distance $\sqrt{2} \approx 1.414$ km. The original code starts at zero distance, compares points to themselves, chooses the maximum distance, and repeats pairs.

Systematic debug procedure

• Run one function at a time using the supplied test data and record the output.

• Change one defect, rerun the same test, and compare the result with the expected output.

• Add boundary tests: empty inputs, one event, equal severities, final-position signature, and pattern longer than the log.

• Run the complete test harness after all fixes to check that optimisation has not changed results.

5. Corrected and optimised production code

This version uses Python's stable Timsort for sorting, a set for repeated blocklist membership, overlapping string search, and divide-and-conquer closest pair. Event coordinates are assumed to be planar x/y values in kilometres.

```
import math

def sort_alerts(alerts):
    """Return alerts by descending severity; preserve equal-severity order."""
    return sorted(alerts, key=lambda alert: alert[1], reverse=True)

def is_blocked(blocklist, ip):
    """Return True when ip is present in the blocklist."""
    return ip in blocklist

def find_signature(log, pattern):
```

```
    """Return every start index, including overlapping matches."""
    if pattern == "":
        return list(range(len(log) + 1))
    hits = []
    start = 0
    while True:
        pos = log.find(pattern, start)
        if pos == -1:
            return hits
        hits.append(pos)
        start = pos + 1

def closest_logins(events):
    """Return ((original_index_a, original_index_b), distance_km).
    For fewer than two points, return (None, infinity).
    Uses divide-and-conquer; ties may return any equally close pair.
    """
    n = len(events)
    if n < 2:
        return None, math.inf

    pts = [(x, y, i) for i, (x, y) in enumerate(events)]
    px = sorted(pts, key=lambda p: (p[0], p[1]))
    py = sorted(pts, key=lambda p: (p[1], p[0]))

    def d2(a, b):
        return (a[0] - b[0])**2 + (a[1] - b[1])**2

    def solve(xs, ys):
        m = len(xs)
        if m <= 3:
            best = math.inf
            pair = None
            for i in range(m):
                for j in range(i + 1, m):
                    value = d2(xs[i], xs[j])
                    if value < best:
                        best, pair = value, (xs[i][2], xs[j][2])
            return best, pair

        mid = m // 2
        left_x, right_x = xs[:mid], xs[mid:]
        split_x = right_x[0][0]
        left_ids = {p[2] for p in left_x}
        left_y = [p for p in ys if p[2] in left_ids]
        right_y = [p for p in ys if p[2] not in left_ids]

        dl, pl = solve(left_x, left_y)
        dr, pr = solve(right_x, right_y)
        best, pair = (dl, pl) if dl <= dr else (dr, pr)

        strip = [p for p in ys if (p[0] - split_x)**2 < best]
        for i in range(len(strip)):
            j = i + 1
            while j < len(strip) and (strip[j][1] - strip[i][1])**2 < best:
                value = d2(strip[i], strip[j])
                if value < best:
                    best, pair = value, (strip[i][2], strip[j][2])
                j += 1
        return best, pair

    best_sq, pair = solve(px, py)
    return pair, math.sqrt(best_sq)


def run_tests():
    alerts = [("A1", 3), ("A2", 9), ("A3", 5),
              ("A4", 9), ("A5", 1)]
    assert sort_alerts(alerts) == [
        ("A2", 9), ("A4", 9), ("A3", 5), ("A1", 3), ("A5", 1)]

    blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
    assert [is_blocked(blocklist, ip) for ip in
            ["10.0.0.9", "10.0.0.5", "8.8.8.8"]] == [True, True, False]
    assert is_blocked([], "8.8.8.8") is False

    assert find_signature("FAILFAIXFAIL", "FAIL") == [0, 8]
    assert find_signature("ABC", "ABCDE") == []
    assert find_signature("AAAA", "AA") == [0, 1, 2]
    assert find_signature("", "") == [0]

    events = [(2, 3), (12, 30), (40, 50),
              (5, 1), (12, 10), (3, 4)]
    pair, dist = closest_logins(events)
    assert set(pair) == {0, 5}
    assert math.isclose(dist, math.sqrt(2), rel_tol=1e-9)
    assert closest_logins([]) == (None, math.inf)
```

```
    assert closest_logins([(1, 2)]) == (None, math.inf)
    print("All tests passed.")

if __name__ == "__main__":
    run_tests()
```

6. Optimisation and complexity analysis

Function	Original / basic method	Optimised method
Sort alerts	Insertion sort: worst-case O(n²); nearly sorted can be O(n).	Stable Timsort: O(n log n) worst-case, O(n) on already/nearly sorted data.
Blocklist	Sequential search: O(B) per query; Q queries O(QB).	Set lookup: expected O(1) per query after O(B) build; Q queries expected O(B+
Signature	Brute-force character matching: O((n–m+1)m), worst-case O(nm).	Built-in find repeatedly: practical efficient search; repeated searches allow overl
Closest pair	Original loops n² distances including self/repeated pairs; incorrect for fewer than two events.	Divide and conquer: O(n log n) time and O(n) auxiliary storage; compares a bou

Let n = number of alerts/log characters/events, B = blocklist size, Q = number of IP queries, and m = pattern length. The original closest-pair code computes n² distances (including n self-pairs and both orders of each distinct pair). A correct unique-pair brute-force version computes n(n–1)/2 distances. For n = 1,000, 4,000, and 8,000, those counts are shown below.

Events n	Original n²	Unique brute force n(n–1)/2
1,000	1,000,000	499,500
4,000	16,000,000	7,998,000
8,000	64,000,000	31,996,000

The divide-and-conquer implementation performs O(n log n) work rather than a fixed n² number of distance calculations. Exact counts depend on the coordinate distribution and strip comparisons, so run the included code with counters or a profiler for measured values. These theoretical counts must not be described as measured timings.

Insertion sort: nearly sorted vs random alerts

Insertion sort is suitable when the list is small or nearly sorted because it can approach O(n) comparisons when only a few shifts are needed. Random or reverse-sorted lists usually require Θ(n²) comparisons and shifts. For about 50,000 alerts, Python's built-in stable sort is generally preferable; it also preserves equal-severity arrival order.

How to measure on the target machine

```
# Suggested measurement procedure:
# 1. Create alert lists of sizes 1,000, 4,000 and 8,000.
# 2. Make one list nearly sorted and another randomly shuffled.
# 3. Count comparisons in a local insertion-sort implementation.
# 4. Add a counter to d2() in brute-force and divide-and-conquer
#    closest-pair implementations, then record counts for each size.
# 5. Run each case several times and report the median elapsed time.
# Report the machine, Python version, input generation and counts.
```

7. Test results expected

Test	Expected result
Sorting sample	A2(9), A4(9), A3(5), A1(3), A5(1)
Blocklist queries	True, True, False
Empty blocklist	False
Signature sample	[0, 8]
Pattern longer than log	[]
Overlapping signature: AAAA / AA	[0, 1, 2]
Closest-pair sample	Indices (0, 5), distance √2 ≈ 1.414 km
0 or 1 login event	(None, infinity)
Full test harness	All tests passed.

8. Conclusion

The defects came from incorrect loop boundaries, early returns, invalid match detection, and reversed closest-distance logic. The corrected module handles the required edge cases. Stable sorting, set membership, overlap-aware signature searching and divide-and-conquer closest-pair analysis improve performance while preserving the intended outputs. Before submission, run the test harness and add genuine machine measurements if required by the instructor.

9. Submission checklist

• Save the code as sentinelops.py and run: python3 sentinelops.py.

• Confirm that the terminal prints “All tests passed.”

• For part (d), collect real comparison/distance counts and elapsed times on your computer; do not invent measurements.

• Commit the final code to your GitHub repository and submit the repository link through Google Classroom, as required by the question.

• Complete the academic-integrity declaration and signature yourself.